

RÉPUBLIQUE TUNISIENNE

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique



[NOM DE L'UNIVERSITÉ / ÉCOLE]

Département d'Informatique

RAPPORT DE PROJET DE FIN D'ÉTUDES

Présenté en vue de l'obtention du

[Diplôme National de Licence Appliquée / Diplôme d'Ingénieur / Mastère Professionnel]

Spécialité : Génie Logiciel et Systèmes Distribués

Conception et Développement de *AegisWeb*

Une Plateforme de Gouvernance et de Confiance
pour les Agents d'Intelligence Artificielle Agissant sur le Web

Réalisé par :

[Prénom NOM Étudiant]

Encadré par :

[Titre. Prénom NOM Encadreur]

Encadreur entreprise :

[Titre. Prénom NOM]

Organisme d'accueil : [Nom de l'entreprise / Projet personnel]

Année

Universitaire : 2025/2026

DÉDICACES

*À mes chers parents,
pour leur amour inconditionnel, leur soutien sans failles
et les innombrables sacrifices qu'ils ont consentis
pour me permettre d'atteindre cet objectif.*

*À ma famille et à mes proches,
dont la présence bienveillante et les encouragements m'ont
accompagné tout au long de ce parcours.*

*À mes camarades et amis,
avec qui j'ai partagé les joies, les défis
et les moments inoubliables de ces années d'études.*

Je vous dédie ce modeste travail.

REMERCIEMENTS

Nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet de fin d'études.

Nos remerciements les plus sincères s'adressent en premier lieu à notre encadreur académique, **[Titre. Prénom NOM]**, pour sa disponibilité permanente, ses orientations rigoureuses, ses précieux conseils et la confiance qu'il nous a accordée tout au long de l'encadrement de ce travail.

Nous remercions également notre encadreur en entreprise, **[Titre. Prénom NOM]**, pour son accueil, le partage de son expertise en cybersécurité et en architecture logicielle, ainsi que les recommandations déterminantes qu'il nous a prodiguées dans la conduite de ce projet.

Nos remerciements s'étendent à l'ensemble du corps enseignant et administratif du **[Nom de l'établissement]**, dont la qualité de l'enseignement dispensé nous a fourni les fondements théoriques et pratiques indispensables à la réalisation de ce travail.

Nous remercions enfin tous nos collègues et amis pour leur soutien moral, leurs suggestions constructives et leur bienveillance constante.

RÉSUMÉ

Dans le cadre de notre Projet de Fin d'Études, nous avons conçu et développé **AegisWeb**, une plateforme de gouvernance et de confiance pour les agents d'intelligence artificielle agissant sur le web. Ce projet répond à un défi émergent des entreprises modernes : comment autoriser des agents IA autonomes à naviguer sur des portails SaaS, télécharger des factures, vérifier des renouvellements et effectuer des changements de plan sans exposer de mots de passe bruts ni permettre d'actions destructrices silencieuses.

Notre solution repose sur une architecture monorepo TypeScript à quatre composants applicatifs : une API de contrôle **NestJS 11**, un worker **BullMQ** avec runtime navigateur **Playwright**, un portail SaaS simulé (*vendor sandbox*), et un tableau de bord opérationnel **Next.js 16**. La persistance est assurée par **PostgreSQL 17** via **Prisma 7**, les files d'attente par **Redis 8**, et le stockage de preuves par **MinIO** (S3-compatible). Le modèle de sécurité intègre un coffre-fort AES-256-GCM, un moteur de politiques pur, un RBAC à cinq rôles, des approbations humaines, une chaîne d'audit SHA-256 et des reçus de confiance (*receipts*).

Les résultats obtenus attestent d'un MVP local complet et fonctionnel, couvrant l'identité des agents, la gestion des fournisseurs SaaS, les credentials chiffrés, les politiques de risque, trois workflows de procurement (téléchargement de facture, vérification de renouvellement, demande de downgrade avec approbation), ainsi qu'un dashboard de preuves opérationnel. La validation repose sur 33 fichiers de tests Vitest (195 tests passés) et des scripts E2E.

Mots-clés : *agents IA, gouvernance, cybersécurité, coffre-fort de secrets, moteur de politiques, approbation humaine, Playwright, NestJS, Next.js, audit trail, procurement SaaS.*

ABSTRACT

As part of our Final Year Project, we designed and developed **AegisWeb**, a governance and trust platform for AI agents acting on the web. This project addresses a critical challenge for modern organizations : how to let autonomous AI agents browse SaaS portals, download invoices, check renewals, and perform plan changes without exposing raw credentials or allowing silent destructive actions.

Our solution is built upon a TypeScript monorepo with four application components : a **NestJS 11** control-plane API, a **BullMQ** worker with **Playwright** browser runtime, a simulated SaaS vendor sandbox, and a **Next.js 16** operational dashboard. Persistence relies on **PostgreSQL 17** via **Prisma 7**, queuing on **Redis 8**, and evidence storage on **MinIO** (S3-compatible). The security model includes an AES-256-GCM vault, a pure policy engine, five-role RBAC, human approvals, SHA-256 audit hash chains, and trust receipts.

The results demonstrate a complete and functional local MVP covering agent identity, SaaS vendor management, encrypted credentials, risk policies, three procurement workflows (invoice download, renewal check, downgrade request with approval), and an evidence-first operational dashboard. Validation relies on 33 Vitest test files (195 passing tests) and E2E scripts.

Keywords : *AI agents, governance, cybersecurity, secret vault, policy engine, human approval, Playwright, NestJS, Next.js, audit trail, SaaS procurement.*

Table des matières

Dédicaces	i
Remerciements	ii
Résumé	iii
Abstract	iv
Liste des figures	viii
Liste des tableaux	x
Liste des abréviations	xii
Introduction Générale	1
1 Cadre du projet	2
1.1 Présentation du contexte et de l'organisme d'accueil	2
1.1.1 Contexte du projet	2
1.1.2 Cadre du projet de fin d'études	3
1.2 Étude de l'existant	3
1.2.1 Présentation des solutions existantes	3
1.2.2 Critique de l'existant	3
1.3 Solution proposée	4
1.3.1 Description d'AegisWeb	4
1.3.2 Flux produit phare Downgrade de plan SaaS	5
1.3.3 Objectifs du projet	5
1.3.4 Périmètre du projet	6
1.4 Méthodologie de gestion de projet	6
1.4.1 Approche incrémentale par phases	6
1.4.2 Planification générale Diagramme de Gantt	7
2 Analyse et spécification des besoins	8
2.1 Identification des acteurs	8
2.2 Besoins fonctionnels	8
2.3 Besoins non fonctionnels	10
2.4 Diagrammes de cas d'utilisation	10

2.4.1	Diagramme de cas d'utilisation global	11
2.4.2	Module Authentification et autorisation	11
2.4.3	Module Workflow et approbation	12
2.5	Descriptions textuelles des cas d'utilisation	12
2.5.1	CU Lancer un workflow de downgrade avec approbation	12
2.5.2	CU Créer et accorder un credential	13
2.6	Backlog produit Phases principales	14
3	Conception du système	16
3.1	Architecture générale du système	16
3.1.1	Architecture logicielle Monorepo	16
3.1.2	Modules NestJS de l'API	17
3.1.3	Architecture physique et déploiement	18
3.2	Modélisation dynamique	19
3.2.1	Diagramme de séquence Authentification	19
3.2.2	Diagramme de séquence Exécution workflow avec approbation	19
3.2.3	Diagramme d'activités Évaluation du moteur de politiques	20
3.3	Modélisation statique	20
3.3.1	Modèle de données Prisma Vue d'ensemble	20
3.3.2	Enums métier principaux	21
3.3.3	Dictionnaire de données Entité WorkflowRun	21
3.3.4	Dictionnaire de données Entité AuditEvent	22
3.3.5	Système de files BullMQ	22
3.3.6	RBAC Matrice des permissions	22
3.3.7	Schéma entité-relation	23
4	Réalisation	24
4.1	Environnement de développement	24
4.1.1	Environnement matériel	24
4.1.2	Environnement logiciel	25
4.2	Technologies utilisées	25
4.2.1	Backend API NestJS	25
4.2.2	Worker et runtime navigateur	26
4.2.3	Frontend Dashboard Next.js	26
4.3	Phases de réalisation et livrables	26
4.3.1	Phases 0–6 Fondation, auth et audit	27
4.3.2	Phases 7–18 Ressources métier et queue	28
4.3.3	Phases 19–29 Worker, workflows et finalisation	29
4.4	Extraits de code significatifs	29

4.4.1	Permissions RBAC partagées	29
4.4.2	Middleware de sécurité frontend	30
4.4.3	Enqueue idempotent BullMQ	30
4.5	Interfaces du dashboard	31
4.5.1	Structure des routes	31
4.5.2	Captures d'interface	32
4.6	Données de démonstration	33
4.7	Tests et validation	33
4.7.1	Stratégie de tests	33
4.7.2	Résultats d'acceptation MVP	34
4.7.3	Tests fonctionnels représentatifs	34
4.7.4	Score audit sécurité	35
4.7.5	Métriques de performance	35
	Conclusion Générale	36
	Bibliographie et Netographie	38
A	Structure du projet Arborescence des fichiers	40
B	Référence des endpoints API	41
C	Variables d'environnement	44
D	Commandes de développement et déploiement	45

Table des figures

1.1	Concept d'identité agent AegisWeb passeport d'autorité contrôlée	5
1.2	Diagramme de Gantt Planning prévisionnel du projet (24 semaines)	7
2.1	Diagramme de cas d'utilisation global du système AegisWeb	11
2.2	Diagramme CU Module Authentification et RBAC	11
2.3	Diagramme CU Module Workflow et approbation	12
3.1	Architecture logicielle globale d'AegisWeb	16
3.2	Architecture de déploiement Docker Compose d'AegisWeb	18
3.3	Diagramme de séquence Authentification JWT + refresh	19
3.4	Diagramme de séquence Workflow downgrade avec approbation	19
3.5	Diagramme d'activités Pipeline d'évaluation des politiques	20
3.6	Schéma entité-relation simplifié base de données AegisWeb	23
4.1	Tableau de bord opérationnel	32
4.2	Détail d'exécution workflow panneau preuves	32
4.3	Gestion des agents IA	32
4.4	Interface d'approbation humaine	32
4.5	Logo principal AegisWeb identité visuelle de la plateforme	33

Liste des tableaux

1.1	Fiche d'identité du projet AegisWeb	2
1.2	Analyse comparative des approches existantes	3
1.3	Principes de la méthodologie par phases	7
2.1	Tableau des acteurs du système AegisWeb	8
2.2	Besoins fonctionnels de la plateforme AegisWeb	9
2.3	Besoins non fonctionnels de la plateforme AegisWeb	10
2.4	Backlog produit Phases principales du développement	14
3.1	Composants applicatifs et bibliothèques partagées	17
3.2	Modules principaux de l'API NestJS	17
3.3	Services conteneurisés et ports locaux	18
3.4	Domaines fonctionnels et entités Prisma	20
3.5	Énumérations métier clés	21
3.6	Dictionnaire de données Modèle WorkflowRun	21
3.7	Dictionnaire de données Modèle AuditEvent	22
3.8	Files BullMQ et types de jobs	22
3.9	Rôles utilisateur et permissions principales	22
4.1	Spécifications de l'environnement matériel	24
4.2	Environnement logiciel et outils de développement	25
4.3	Technologies du backend AegisWeb	25
4.4	Technologies du worker et browser-runtime	26
4.5	Technologies du frontend AegisWeb	26
4.6	Phases 0 à 6 Fondations et sécurité transversale	27
4.7	Phases 7 à 18 Modules CRUD et orchestration	28
4.8	Phases 19 à 29 Worker, browser et workflows SaaS	29
4.9	Routes principales du dashboard (/app/*)	31
4.10	Données seed Organisation Northstar Labs	33
4.11	Résultats de la suite d'acceptation MVP backend	34
4.12	Tests fonctionnels backend représentatifs	34
4.13	Scores d'audit sécurité (docs/SECURITY_AUDIT.md)	35
4.14	Métriques mesurées en environnement de développement local	35

B.1	Référence des endpoints API AegisWeb	41
C.1	Variables d'environnement principales (<code>.env.example</code>)	44
D.1	Commandes pnpm principales	45
D.2	URLs de services en développement local	45

LISTE DES ABRÉVIATIONS

Abréviation	Signification
AES-GCM	Advanced Encryption Standard Galois/Counter Mode
AI / IA	Artificial Intelligence / Intelligence Artificielle
API	Application Programming Interface
Argon2id	Algorithme de hachage de mots de passe
BFF	Backend For Frontend
BullMQ	File de messages Redis pour Node.js
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSP	Content Security Policy
DI	Dependency Injection
DTO	Data Transfer Object
GCM	Galois/Counter Mode (mode de chiffrement)
HMAC	Hash-based Message Authentication Code
HTTP/S	Hypertext Transfer Protocol (Secure)
IDE	Integrated Development Environment
JWT	JSON Web Token
MVP	Minimum Viable Product
ORM	Object-Relational Mapping
PFE	Projet de Fin d'Études
RBAC	Role-Based Access Control
REST	Representational State Transfer
RPA	Robotic Process Automation
S3	Simple Storage Service (stockage objet)
SaaS	Software as a Service
SDK	Software Development Kit
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
TLS	Transport Layer Security
TTL	Time To Live
UML	Unified Modeling Language
URL	Uniform Resource Locator
UUID	Universally Unique Identifier

INTRODUCTION GÉNÉRALE

L'émergence des agents d'intelligence artificielle capables d'agir de manière autonome sur le web transforme profondément les processus opérationnels des entreprises. Ces agents peuvent désormais naviguer sur des portails SaaS, remplir des formulaires, télécharger des factures et modifier des paramètres de facturation. Pourtant, le goulot d'étranglement n'est plus seulement l'intelligence : c'est la **permission**.

Les entreprises ne peuvent pas confier à des agents autonomes des mots de passe bruts, un accès administrateur illimité, une autorité de paiement non contrôlée ou un contrôle navigateur sans garde-fous. Les solutions RPA traditionnelles et les intégrations API classiques ne couvrent pas le besoin de gouvernance fine, d'approbation humaine contextuelle et de preuves vérifiables pour chaque action sensible.

Problématique : Comment concevoir et développer une plateforme de gouvernance pour agents IA web, offrant des identités dédiées, un coffre-fort de credentials, un moteur de politiques, des approbations humaines, une exécution navigateur contrôlée et un audit traçable, tout en répondant aux exigences de sécurité, de performance et de maintenabilité d'un système distribué moderne ?

Organisation du rapport : Ce rapport s'articule autour de quatre chapitres complémentaires :

- **Chapitre 1 Cadre du projet :** présentation du contexte, analyse de l'existant, description de la solution AegisWeb et méthodologie de développement incrémental.
- **Chapitre 2 Analyse et spécification des besoins :** identification des acteurs, besoins fonctionnels et non fonctionnels, cas d'utilisation et backlog produit.
- **Chapitre 3 Conception du système :** architecture monorepo, modélisations dynamique et statique, dictionnaire de données.
- **Chapitre 4 Réalisation :** environnement, technologies, livrables par phase, interfaces et résultats des tests.

CHAPITRE 1

CADRE DU PROJET

Ce premier chapitre situe notre projet dans son contexte professionnel et technique. Nous y présentons le domaine de la gouvernance des agents IA, analysons l'existant pour en identifier les lacunes, décrivons la solution AegisWeb, puis exposons la méthodologie et le planning retenus.

1.1 Présentation du contexte et de l'organisme d'accueil

1.1.1 Contexte du projet

AegisWeb est une plateforme de **gouvernance et de confiance pour agents IA agissant sur le web**. Elle fournit une couche de contrôle entre les agents autonomes et les systèmes métier (portails SaaS, formulaires, facturation, etc.). Son principe fondateur est *à Permission before autonomy* : la permission est le goulot d'étranglement, pas l'intelligence.

Le premier cas d'usage ciblé est le **procurement SaaS** : permettre à des agents IA de gérer en toute sécurité les factures, les renouvellements et les changements de plan soumis à approbation humaine, sans exposer de credentials bruts.

TABLE 1.1 – Fiche d'identité du projet AegisWeb

Critère	Description
Dénomination	AegisWeb (nom interne historique : AgentPass)
Secteur d'activité	Cybersécurité applicative, gouvernance IA
Nature	Plateforme MVP locale de gouvernance d'agents web
Produit phare	Couche permission, credential, approbation, runtime navigateur et audit pour agents IA
Marché cible	Entreprises utilisant des agents IA pour l'automatisation web contrôlée
Cas d'usage MVP	Procurement SaaS (factures, renouvellements, downgrades avec approbation)
Langues de l'interface	Anglais (dashboard), documentation bilingue

1.1.2 Cadre du projet de fin d'études

Notre projet de fin d'études consiste en la **conception et le développement complets de la plateforme AegisWeb**, couvrant :

- le **backend** (API REST NestJS, base de données, files BullMQ, endpoints internes worker) ;
- le **worker** (exécution de workflows, Playwright, connecteur vendor) ;
- le **frontend** (dashboard Next.js, authentification, couche données React Query) ;
- les **bibliothèques partagées** (domain, vault, policy-engine, browser-runtime, audit) ;
- l'**infrastructure** locale (Docker Compose, PostgreSQL, Redis, MinIO, Mailpit).

1.2 Étude de l'existant

1.2.1 Présentation des solutions existantes

TABLE 1.2 – Analyse comparative des approches existantes

Solution	Points forts	Limitations	Gouvernance IA
RPA classique (UiPath, etc.)	Automatisation UI mature, intégrations entreprise	Pas d'identité agent, pas de politiques fines, audit limité	Faible
Intégrations API directes	Rapides, déterministes, testables	Couverture partielle des portails, pas d'actions UI	Moyenne
Gestionnaires de secrets (Vault)	Chiffrement robuste, rotation, audit	Pas de runtime navigateur, pas d'approbation workflow	Partielle
Agents IA autonomes (bruts)	Flexibilité maximale, intelligence adaptative	Risque credential, actions silencieuses, pas de preuves	Inexistante
AegisWeb (proposé)	Identité agent, vault, politiques, approbation, preuves, audit	MVP local, pas en-core SaaS production multi-tenant	Élevée

1.2.2 Critique de l'existant

L'analyse révèle plusieurs lacunes fondamentales :

- **Absence d'identité non-humaine dédiée** pour les agents, avec statut, purpose et grants explicites.
- **Exposition des credentials** aux agents ou aux prompts LLM, sans chiffrement au repos ni déchiffrement scopé par exécution.
- **Absence de moteur de politiques** évaluant domaines, actions interdites, seuils financiers et niveaux de risque.
- **Pas de pause pour approbation humaine** avant les actions destructrices (downgrade, annulation).
- **Manque de preuves opérationnelles** : screenshots, factures, reçus de confiance et chaîne d'audit vérifiable.
- **Runtime navigateur non contrôlé** : accès réseau privé, popups hors domaine, téléchargements non tracés.

1.3 Solution proposée

1.3.1 Description d'AegisWeb

AegisWeb est la couche de permission, credential, approbation, runtime navigateur et audit pour les agents IA agissant sur le web. Elle donne aux agents une **autorité contrôlée** via :

- **Identité agent** : agent non-humain avec identifier unique, statut (active, paused, revoked) et purpose documenté.
- **Coffre-fort de credentials** : chiffrement AES-256-GCM au repos, octroi par agent via `CredentialAgentGrant`.
- **Moteur de politiques** : allowlist de domaines, actions interdites, seuils de dépense, règles d'approbation.
- **Runtime navigateur contrôlé** : Playwright avec allowlist, blocage réseau privé, masquage des champs sensibles.
- **Workflows queue-backed** : trois templates SaaS procurement.
- **Approbation humaine** : pause du run, notification email, reprise conditionnelle.
- **Audit trail** : événements append-only avec chaîne de hash SHA-256 (`prevHash + eventHash`).
- **Receipts** : artefacts finaux de confiance résumant timeline, décisions et preuves.
- **Dashboard opérationnel** : salle de preuves pour agents, vendors, politiques, runs, approvals, audit.

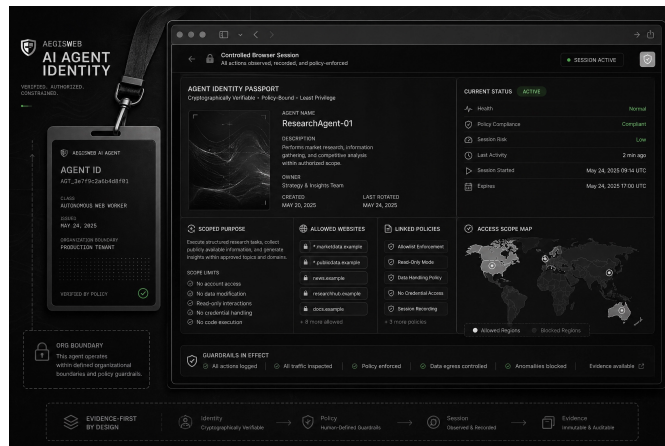


FIGURE 1.1 – Concept d'identité agent AegisWeb passeport d'autorité contrôlée

1.3.2 Flux produit phare Downgrade de plan SaaS

1. L'owner se connecte au dashboard.
2. Il crée ou sélectionne agent, vendor, credential, policy, workflow.
3. Il lance un Acme Downgrade Request.
4. L'API crée un WorkflowRun et enqueue un job BullMQ.
5. Le worker ouvre un navigateur contrôlé (Playwright).
6. Le worker se connecte au sandbox via credential déchiffré (API interne vault, scopé au run).
7. Le worker lit les données de renouvellement et de plan.
8. Le moteur de politique classe l'action comme risquée → **pause pour approbation**.
9. L'approbateur examine les preuves (screenshots) et approuve ou rejette.
10. Le worker reprend uniquement après approbation.
11. L'action est soumise → **receipt + audit trail** générés.
12. Notification email via Mailpit (en local).

1.3.3 Objectifs du projet

Objectifs fonctionnels :

- Développer une API REST NestJS avec 20+ modules (auth, RBAC, agents, vendors, policies, credentials, workflows, runs, approvals, receipts, audit, files, queue, internal worker).
- Implémenter un schéma Prisma multi-tenant (17 tables) avec seed de démonstration riche.
- Concevoir un moteur de politiques pur TypeScript sans dépendance BDD.
- Réaliser un coffre-fort AES-256-GCM avec redaction récursive des secrets.

- Développer un worker BullMQ + Playwright avec trois workflows SaaS.
- Mettre en place un dashboard Next.js 16 evidence-first.
- Fournir un vendor sandbox pour démos et tests déterministes.

Objectifs non fonctionnels :

- Isolation multi-tenant stricte par `organizationId`.
- Argon2id, JWT courte durée, refresh `HttpOnly` atomique.
- Rate limiting Redis en production.
- CSP nonce-based sans `unsafe-eval` sur le frontend.
- Tests d'intégration Vitest couvrant les 30 phases backend.
- Déploiement Docker Compose reproductible.

1.3.4 Périmètre du projet

Le périmètre couvre :

1. Le monorepo `agentpass` (`pnpm workspaces`).
2. Les applications `api`, `worker`, `web`, `vendor-sandbox`.
3. Les bibliothèques `domain`, `database`, `vault`, `policy-engine`, `browser-runtime`, `audit`, `testing`.
4. L'infrastructure `infra/docker-compose.yml`.
5. 33 fichiers de tests Vitest + scripts `smoke/E2E`.

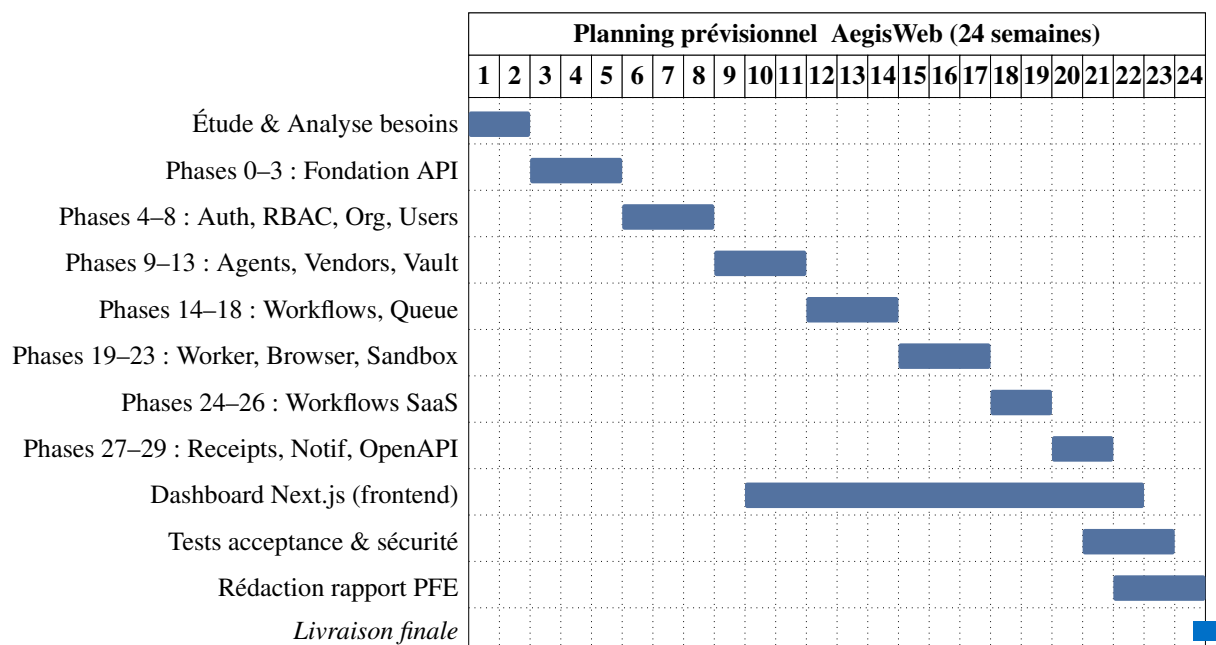
1.4 Méthodologie de gestion de projet**1.4.1 Approche incrémentale par phases**

Contrairement à une approche SCRUM classique par sprints, AegisWeb a été développé selon une **méthodologie incrémentale en 30 phases** (documentées dans `docs/BACKEND_IMPLEMENTATION`, chaque phase livrant un module testable et validé avant passage à la suivante.

TABLE 1.3 – Principes de la méthodologie par phases

Élément	Application dans le projet AegisWeb
Phase	Incrément fonctionnel avec tests dédiés (<code>phaseN.spec.ts</code>)
Definition of Done	Tests passants, module intégré, notes de complétion documentées
Acceptance MVP	<code>mvp-backend-acceptance.spec.ts</code> (31 fichiers, 195 tests)
Traçabilité	<code>docs/PHASE*_COMPLETION_NOTES.md</code> (27 fichiers)
Sécurité continue	<code>security-coverage.spec.ts</code> + <code>docs/SECURITY_AUDIT.md</code>

1.4.2 Planification générale Diagramme de Gantt

FIGURE 1.2 – Diagramme de Gantt Planning prévisionnel du projet (24 semaines)

Conclusion du chapitre : Ce chapitre nous a permis de situer AegisWeb dans le contexte de la gouvernance des agents IA web, d'identifier les insuffisances des solutions existantes et de définir les contours de notre plateforme. La méthodologie incrémentale en 30 phases et le planning sur 24 semaines constituent le cadre organisationnel de notre démarche. Le chapitre suivant sera consacré à l'analyse détaillée des besoins.

CHAPITRE 2

ANALYSE ET SPÉCIFICATION DES BESOINS

Ce chapitre est dédié à l'analyse des besoins de la plateforme AegisWeb. Nous y identifions les acteurs, détaillons les besoins fonctionnels et non fonctionnels, présentons les diagrammes de cas d'utilisation et définissons le backlog produit initial.

2.1 Identification des acteurs

TABLE 2.1 – Tableau des acteurs du système AegisWeb

Acteur	Type	Rôle et description
Owner / Admin	Primaire	Propriétaire de l'organisation. Gère agents, vendors, politiques, credentials, workflows. Lance et annule des runs.
Approver	Primaire	Examine les demandes d'approbation, consulte les preuves (screenshots, audit) et approuve ou rejette les actions risquées.
Developer	Primaire	Développeur technique. Consulte les ressources et peut lancer/annuler des workflows (<code>workflow:run</code> , <code>workflow:cancel</code>).
Auditor	Primaire	Auditeur de conformité. Accès lecture sur agents, runs, receipts, audit trail et fichiers de preuve.
Agent IA	Secondaire	Identité non-humaine exécutant des workflows via le worker, soumise aux politiques et grants de credentials.
Worker	Secondaire	Processus BullMQ consommant les jobs, exécutant Playwright et appelant l'API interne avec token HMAC scopé.
Vendor Sandbox	Secondaire	Portail SaaS simulé (facturation, factures, downgrade) pour démos et tests sans credentials réels.
Système (Mailpit/S3)	Secondaire	Services d'infrastructure : stockage objet MinIO, notifications SMTP Mailpit, Redis BullMQ.

2.2 Besoins fonctionnels

TABLE 2.2 – Besoins fonctionnels de la plateforme AegisWeb

ID	Module	Description
BF01	Auth.	Inscription organisation, login Argon2id, JWT access token, refresh token HttpOnly (agentpass_refresh_token), refresh atomique, logout.
BF02	RBAC	Cinq rôles (owner, admin, approver, auditor, developer) avec 28 permissions granulaires et isolation organizationId.
BF03	Agents	CRUD agents non-humains (identifier, purpose, status, grants credentials).
BF04	Vendors	Gestion fournisseurs SaaS (website, catégorie, coût mensuel estimé).
BF05	Policies	Politiques JSON typées (allowlist, actions, spending, approval rules, bundles).
BF06	Vault	Credentials chiffrés AES-256-GCM, octroi/révocation par agent, déchiffrement scopé run.
BF07	Workflows	Définitions réutilisables (3 templates) : invoice download, renewal check, plan downgrade.
BF08	Runs	Exécutions workflow avec états (queued, running, waiting_for_approval, completed, failed, canceled, denied).
BF09	Policy Eng.	Évaluation pure : agent actif, domaine, allowlist, actions interdites, seuils, approbation, risque.
BF10	Browser RT	Playwright contrôlé : allowlist domaines, blocage réseau privé, screenshots masqués, downloads SHA-256.
BF11	Approvals	Pause run, création ApprovalRequest, notification email, approve/reject, reprise queue.
BF12	Audit	Événements typés append-only, chaîne hash SHA-256, acteurs (user, agent, worker, system).
BF13	Files	Métadonnées fichiers S3 (screenshots, factures PDF, traces) avec isolation org.
BF14	Receipts	Artefacts finaux de confiance (timeline, décisions, approbation, preuves).
BF15	Dashboard	Interface Next.js pour toutes les ressources + panneaux evidence (timeline, screenshots, audit drawer).

ID	Module	Description
BF16	Sandbox	Portail SaaS factice (login, billing, invoices, downgrade, cancel, admin users).
BF17	Queue	BullMQ : workflow-runs, workflow-resume, workflow-maintenance avec jobs idempotents.
BF18	OpenAPI	Documentation Swagger activable via <code>ENABLE_OPENAPI=true</code> .

2.3 Besoins non fonctionnels

TABLE 2.3 – Besoins non fonctionnels de la plateforme AegisWeb

Critère	Exigence
Sécurité	Argon2id, JWT courte durée, refresh HttpOnly atomique, AES-256-GCM vault, tokens worker HMAC, CSP nonce, rate limiting Redis, redaction secrets dans audit/receipts.
Isolation tenant	Toutes les requêtes API scopées par <code>organizationId</code> . Tests régression cross-org.
Traçabilité	Audit append-only avec <code>prevHash</code> et <code>eventHash</code> . Receipts immuables post-exécution.
Performance	Worker concurrency = 1 par défaut. Retry BullMQ 3 tentatives, backoff exponentiel.
Maintenabilité	Monorepo TypeScript, libs partagées, Prisma migrations, 33 specs Vitest, ESLint 9.
Disponibilité	Endpoints <code>/health</code> et <code>/health/ready</code> avec vérification Postgres/Redis.
Déployabilité	Docker multi-stage (api, worker, web, vendor-sandbox), profils infra et app, guide Render.
UX Dashboard	Design evidence-first, dense, monochrome, orienté confiance. Aucun secret en clair dans l'UI.
Pilot readiness	Score audit sécurité documenté : 3,8/5 global (<code>docs/SECURITY_AUDIT.md</code>).

2.4 Diagrammes de cas d'utilisation

2.4.1 Diagramme de cas d'utilisation global

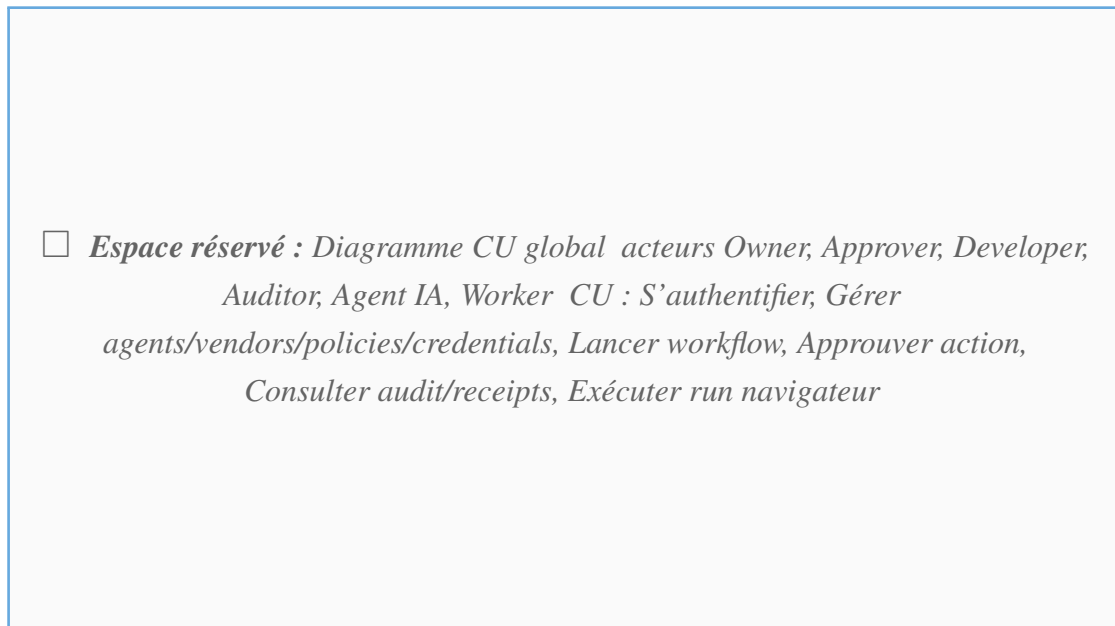


FIGURE 2.1 – Diagramme de cas d'utilisation global du système AegisWeb

2.4.2 Module Authentification et autorisation

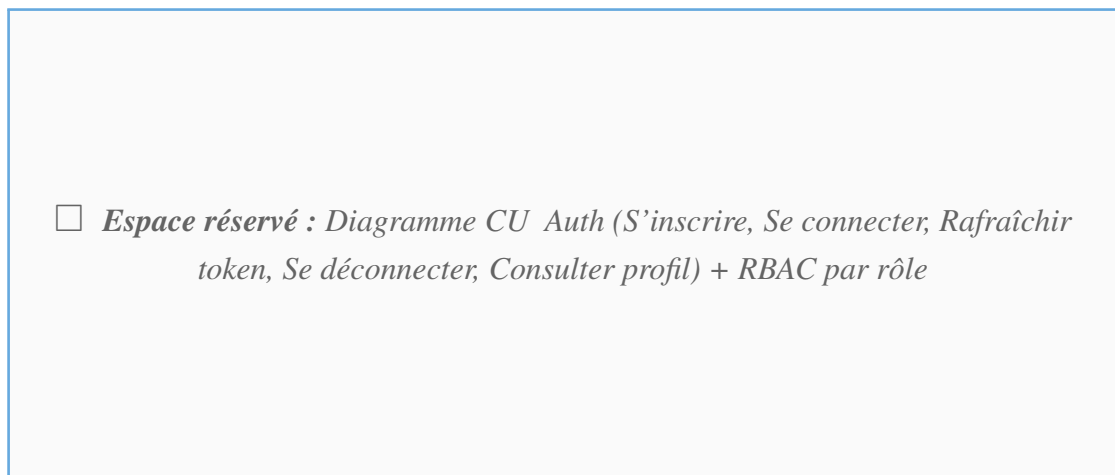


FIGURE 2.2 – Diagramme CU Module Authentification et RBAC

2.4.3 Module Workflow et approbation

□ *Espace réservé : Diagramme CU Workflow (Créer workflow, Lancer run, [extend] Demander approbation, Approuver/Rejeter, Reprendre run, Générer receipt) acteurs Owner, Approver, Worker*

FIGURE 2.3 – Diagramme CU Module Workflow et approbation

2.5 Descriptions textuelles des cas d'utilisation

2.5.1 CU Lancer un workflow de downgrade avec approbation

Cas d'utilisation CU-WF-01 : Lancer un downgrade SaaS avec approbation

Acteur principal Owner ou Developer

Acteurs secondaires Worker, Policy Engine, Approver, Mailpit

Pré-condition Agent actif, credential accordé, policy active, workflow plan_downgrade_request configuré

Post-condition Run completed ou denied, receipt et audit générés

Déclencheur Clic ñ Run workflow ž sur le dashboard

Scénario nominal :

1. L'utilisateur authentifié lance le workflow depuis le dashboard.
2. L'API crée un WorkflowRun (statut queued) et enqueue un job start-{runId}.
3. Le worker consomme le job, passe le run en running.
4. Le worker ouvre un contexte Playwright isolé (allowlist vendor).
5. Le worker demande le déchiffrement du credential via POST /internal/vault/credentials/:id/decrypt-for-run.
6. Le worker se connecte au vendor sandbox et lit les données billing.

7. Chaque action est enregistrée comme `ActionAttempt` avec décision du policy engine.
8. Pour `change_plan` : décision `require_approval` → `run waiting_for_approval`.
9. Email envoyé à l'approuver via Mailpit.
10. L'approuver examine screenshots et approuve.
11. Job `resume-{runId}-{approvalId}` enqueue.
12. Le worker soumet le downgrade et génère receipt + audit.

Scénarios alternatifs :

- **7a.** Décision `deny` → `run denied`, pas de soumission.
- **10a.** Rejet approbation → `run denied`.
- **E1.** Credential révoqué → `run failed`.
- **E2.** Domaine hors allowlist → action bloquée par browser-runtime.

2.5.2 CU Créer et accorder un credential

Cas d'utilisation CU-VAULT-01 : Créer et accorder un credential

Acteur principal Owner ou Admin

Acteurs secondaires Vault (AES-256-GCM), Audit

Pré-condition Vendor existant, permission `credential:create`

Post-condition Credential chiffré créé, grant agent optionnel, événement audit enregistré

Déclencheur Formulaire "New credential" sur le dashboard

Scénario nominal :

1. L'utilisateur saisit le type (username/password, API token, etc.) et le payload secret.
2. L'API chiffre le payload via `encryptSecret` (vault lib).
3. Persistance Prisma : seul le ciphertext est stocké.
4. L'utilisateur accorde l'accès à un agent (`CredentialAgentGrant`).
5. Événements `credential_created` et `credential_granted_to_agent` dans l'audit trail.
6. L'UI affiche les métadonnées sans jamais exposer le secret en clair.

2.6 Backlog produit Phases principales

TABLE 2.4 – Backlog produit Phases principales du développement

ID	Livrable	Priorité	Phase	Statut
P01	Fondation API, worker boot, libs partagées	Must Have	0	✓
P02	Schéma Prisma + seed démo Northstar Labs	Must Have	1	✓
P03	Auth Argon2id + JWT + refresh HttpOnly	Must Have	4	✓
P04	RBAC 5 rôles + isolation organisation	Must Have	5	✓
P05	Audit append-only + hash chain	Must Have	6	✓
P06	Files S3/MinIO + métadonnées	Must Have	7	✓
P07	Agents, Vendors, Policies CRUD	Must Have	9–12	✓
P08	Vault AES-GCM + Credentials-Module	Must Have	13	✓
P09	Workflows + WorkflowRuns + Queue BullMQ	Must Have	14–18	✓
P10	Worker + Browser Runtime Playwright	Must Have	19–20	✓
P11	Vendor Sandbox + Connector	Must Have	21–22	✓
P12	3 workflows SaaS (invoice, renewal, downgrade)	Must Have	24–26	✓
P13	Approvals + Notifications email	Must Have	17, 28	✓
P14	Receipts + preuves finales	Must Have	27	✓
P15	Dashboard Next.js complet	Must Have		✓
P16	OpenAPI / Swagger	Should Have	29	✓
P17	Tests acceptance MVP (195 tests)	Must Have		✓

Conclusion du chapitre : Ce chapitre a permis d'identifier les 8 catégories d'acteurs du système AegisWeb et de formaliser 18 besoins fonctionnels et 9 critères non fonctionnels. Le backlog de 17 livrables majeurs, aligné sur les 30 phases de développement, constitue la feuille de route de notre implémentation. Le chapitre suivant sera consacré à la conception architecturale et à la modélisation détaillée du système.

CHAPITRE 3

CONCEPTION DU SYSTÈME

Ce chapitre décrit la conception architecturale et la modélisation du système AegisWeb. Nous y présentons l'architecture monorepo, les modélisations dynamiques et statiques, ainsi que le dictionnaire de données Prisma.

3.1 Architecture générale du système

3.1.1 Architecture logicielle Monorepo

AegisWeb repose sur une **architecture monorepo TypeScript** (pnpm workspaces) avec séparation claire entre applications et bibliothèques partagées.

□ *Espace réservé : Schéma d'architecture AegisWeb* Utilisateur/Approver → Next.js Web (3000) → NestJS API (3001) → PostgreSQL + Redis/BullMQ + MinIO/S3 Worker → Playwright → Vendor Sandbox (4202)

FIGURE 3.1 – Architecture logicielle globale d'AegisWeb

TABLE 3.1 – Composants applicatifs et bibliothèques partagées

Composant	Rôle	Technologies
apps/web	Dashboard + marketing, auth session	Next.js 16, React 19, Tailwind 4
apps/api	Control plane REST, queue, internal API	NestJS 11, Prisma 7, BullMQ
apps/worker	Exécution workflows, Playwright	NestJS context, BullMQ
apps/vendor-saas	Portail SaaS simulé	NestJS minimal
libs/domain	Enums, permissions, types queue/workflow	TypeScript pur
libs/vault	Chiffrement AES-256-GCM, redaction	Node.js crypto
libs/policy-engine	Évaluation politiques/risk	TypeScript pur
libs/browser-wrapper	Wrapper Playwright contrôlé	Playwright Chromium

3.1.2 Modules NestJS de l'API

L'API (apps/api/src/app.module.ts) importe 20+ modules :

TABLE 3.2 – Modules principaux de l'API NestJS

Module	Responsabilité
AuthModule	Login, register, refresh, logout (Argon2id, JWT, cookies)
AuthorizationModule	JwtAuthGuard, PermissionsGuard, isolation org
AgentsModule	CRUD identités agents non-humaines
VendorsModule	Fournisseurs SaaS
PoliciesModule	Politiques + évaluation via policy-engine
CredentialsModule	Coffre-fort, grants, métadonnées
WorkflowsModule	Définitions de workflows (templates)
WorkflowRunsModule	Exécutions, état, annulation
ApprovalsModule	Cycle de vie approbations humaines
QueueModule	Enqueue BullMQ (start, resume, cancel)
InternalWorkerModule	Endpoints worker (events, screenshots, vault decrypt)
AuditModule	Événements audit + chaîne hash SHA-256
ReceiptsModule	Artefacts finaux de confiance
FilesModule	Métadonnées + stockage S3

Middleware global (ordre) :

1. RequestContextMiddleware user, org, request ID
2. SecurityHeadersMiddleware
3. RateLimitMiddleware Redis TTL en production
4. RequestLoggingMiddleware

3.1.3 Architecture physique et déploiement

□ *Espace réservé : Docker Compose postgres :17 (5432), redis :8 (6379), minio (9000/9001), mailpit (1025/8025), api (3001), worker, web (3000), vendor-sandbox (4202), migrate profils infra et app*

FIGURE 3.2 – Architecture de déploiement Docker Compose d'AegisWeb**TABLE 3.3** – Services conteneurisés et ports locaux

Service	Image/Port	Rôle
postgres	postgres :17 / 5432	Base agentpass
redis	redis :8 / 6379	BullMQ + rate limit
minio	minio / 9000, 9001	Stockage S3-compatible
mailpit	axllent/mailpit	SMTP 1025, UI 8025
api	NestJS / 3001	API REST + Swagger
worker	NestJS + Playwright	Consommation jobs
web	Next.js / 3000	Dashboard
vendor-sandbox	NestJS / 4202	Portail SaaS factice

3.2 Modélisation dynamique

3.2.1 Diagramme de séquence Authentification

□ *Espace réservé : Séquence Auth Client → POST /auth/login → AuthService (Argon2id verify) → JWT access (body) + refresh (cookie HttpOnly) → cookie aegisweb_session (middleware) branche 401 → /auth/refresh*

FIGURE 3.3 – Diagramme de séquence Authentification JWT + refresh

3.2.2 Diagramme de séquence Exécution workflow avec approbation

□ *Espace réservé : Séquence Workflow Owner → POST /workflow-runs → QueueService.enqueue → Worker → Playwright → PolicyEngine.evaluate → [require_approval] → ApprovalRequest → Email → Approver → POST /approvals/:id/approve → resume queue → Receipt*

FIGURE 3.4 – Diagramme de séquence Workflow downgrade avec approbation

3.2.3 Diagramme d'activités Évaluation du moteur de politiques

□ *Espace réservé : Activités Policy Engine* *ActionAttempt reçue* → *Agent actif?* → *Domaine bloqué?* → *Allowlist OK?* → *Action interdite?* → *Seuil montant?* → *[allow | deny | require_approval | pause_agent] + RiskLevel + signaux*

FIGURE 3.5 – Diagramme d'activités Pipeline d'évaluation des politiques

3.3 Modélisation statique

3.3.1 Modèle de données Prisma Vue d'ensemble

Le schéma (prisma/schema.prisma) comprend **17 modèles** principaux et de nombreux enums typés.

TABLE 3.4 – Domaines fonctionnels et entités Prisma

Domaine	Modèles / Entités
Identité	Organization, User, RefreshToken
Agents	Agent
Fournisseurs	Vendor
Politiques	Policy
Credentials	Credential, CredentialAgentGrant
Workflows	Workflow, WorkflowRun, ActionAttempt
Approbations	ApprovalRequest
Audit & Preuves	AuditEvent, File, Receipt

3.3.2 Enums métier principaux

TABLE 3.5 – Énumérations métier clés

Enum	Valeurs principales
UserRole	owner, admin, approver, auditor, developer
WorkflowTemplate	vendor_invoice_download, saas_renewal_check, plan_downgrade_request
WorkflowRunStat	queued, running, waiting_for_approval, completed, failed, canceled, denied
PolicyDecision	allow, deny, require_approval, require_step_up_auth, pause_agent
RiskLevel	low, medium, high, critical
ActionType	open_page, read_page, fill_form, click_button, download_file, change_plan, cancel_subscription, ...

3.3.3 Dictionnaire de données Entité WorkflowRun

TABLE 3.6 – Dictionnaire de données Modèle WorkflowRun

Attribut	Type	Description
id	UUID	Clé primaire
organizationId	UUID (FK)	Isolation multi-tenant
workflowId	UUID (FK)	Définition parente
agentId	UUID (FK)	Agent exécutant
status	WorkflowRunStatus	État courant du run
stateJson	JSON	État interne sérialisé (étapes, contexte)
requestedByUserId	UUID	Utilisateur ayant lancé le run
startedAt	/ DateTime	Horodatages
completedAt		
failureReason	String ?	Raison d'échec si failed

3.3.4 Dictionnaire de données Entité AuditEvent

TABLE 3.7 – Dictionnaire de données Modèle AuditEvent

Attribut	Type	Description
id	UUID	Clé primaire
organizationId	UUID (FK)	Tenant
eventType	AuditEventType	Type typé (50+ événements)
actorType	AuditActorType	user, agent, worker, system
actorId	String ?	Identifiant de l'acteur
prevHash	String	Hash événement précédent (chaîne)
eventHash	String	SHA-256 de l'événement courant
payloadJson	JSON	Données redactées (secrets masqués)
createdAt	DateTime	Horodatage append-only

3.3.5 Système de files BullMQ

TABLE 3.8 – Files BullMQ et types de jobs

Queue	Usage	ID job
workflow-runs	Démarrage de workflows	start-{runId}
workflow-resume	Reprise après approbation	resume-{runId}-{appro
workflow-maint	Signaux d'annulation	cancel-{runId}

3.3.6 RBAC Matrice des permissions

TABLE 3.9 – Rôles utilisateur et permissions principales

Rôle	Permissions
owner / admin	Toutes (28 permissions)
approver	Lecture + approval:approve
auditor	Lecture + audit:read + file:read
developer	Lecture + workflow:run + workflow:cancel

3.3.7 Schéma entité-relation

□ *Espace réservé : ER simplifié Organization (1)–(*) User, Agent, Vendor, Policy, Workflow Agent (*)–(*) Credential via Grant Workflow (1)–(*) WorkflowRun (1)–(*) ActionAttempt, ApprovalRequest, Receipt WorkflowRun (1)–(*) AuditEvent, File*

FIGURE 3.6 – Schéma entité-relation simplifié base de données AegisWeb

Conclusion du chapitre : La conception d’AegisWeb révèle une architecture modulaire et sécurisée : monorepo TypeScript, séparation control plane / worker, moteur de politiques pur, vault chiffré et modèle de données Prisma riche en 17 tables. Ces fondements guident notre implémentation, décrite dans le chapitre suivant.

CHAPITRE 4

RÉALISATION

Ce chapitre présente la réalisation technique de la plateforme AegisWeb. Après la description de l'environnement et des technologies, nous exposons les livrables des phases principales, illustrons les interfaces du dashboard et présentons les résultats des tests de validation.

4.1 Environnement de développement

4.1.1 Environnement matériel

TABLE 4.1 – Spécifications de l'environnement matériel

Composant	Spécification
Processeur	Intel Core i7 / AMD Ryzen 7 (multi-curs)
Mémoire RAM	16 Go minimum (32 Go recommandé pour Playwright)
Stockage	SSD NVMe 512 Go
Système d'exploitation	Linux (Ubuntu) / Windows 11 + WSL2
Connexion réseau	Internet pour installation dépendances npm

4.1.2 Environnement logiciel

TABLE 4.2 – Environnement logiciel et outils de développement

Catégorie	Outil / Version
Runtime	Node.js ≥ 22
Gestionnaire paquets	pnpm ≥ 10 (workspaces)
Éditeur	Visual Studio Code / Cursor
Langage	TypeScript 5.7+
Conteneurisation	Docker Compose v2
Base de données	PostgreSQL 17, Prisma 7.8
Tests	Vitest 2.1 + Supertest
Lint	ESLint 9
Navigateur automatisé	Playwright Chromium (worker)
Client API	Swagger (/docs), Bruno/Postman

4.2 Technologies utilisées

4.2.1 Backend API NestJS

TABLE 4.3 – Technologies du backend AegisWeb

Technologie	Version	Rôle
Node.js	22+	Runtime JavaScript serveur
NestJS	11	Framework modulaire (DI, guards)
Prisma	7.8	ORM, migrations, client typé
PostgreSQL	17	Base relationnelle
Redis + BullMQ	8 / 5	Files de workflows
AWS SDK (S3)	3.x	Stockage MinIO/S3
Argon2		Hachage mots de passe
jsonwebtoken		JWT access tokens
Zod	3.24+	Validation schémas
Vitest + Supertest	2.1+	Tests intégration API

4.2.2 Worker et runtime navigateur

TABLE 4.4 – Technologies du worker et browser-runtime

Technologie	Version	Rôle
NestJS application context	11	Boot worker sans HTTP public
BullMQ Worker	5	Consommation jobs (concurrency=1)
Playwright	1.60	Automatisation Chromium
libs/browser-runtime		Allowlist, screenshots, downloads
libs/vault		Déchiffrement credential scopé run
HMAC worker tokens		Auth endpoints internes (6h TTL)

4.2.3 Frontend Dashboard Next.js

TABLE 4.5 – Technologies du frontend AegisWeb

Technologie	Version	Rôle
Next.js	16	App Router, SSR, middleware
React	19	Composants UI
Tailwind CSS	4	Styles utilitaires
Radix UI / shadcn		Primitives accessibles (components/ui)
TanStack Query	React 5	Cache serveur, polling runs/approvals
react-hook-form Zod	+	Formulaires typés
Sonner		Notifications toast
Iconoir + Lucide		Iconographie

4.3 Phases de réalisation et livrables

4.3.1 Phases 0–6 Fondation, auth et audit

TABLE 4.6 – Phases 0 à 6 Fondations et sécurité transversale

Ph.	Livrable	Tests	Statut
0	Boot API, worker, sandbox, libs	phase0.spec.ts	✓
1	Prisma schema + seed Northstar Labs	phase1-database	✓
2	Lib domain (enums, permissions)	phase2-domain	✓
3	Config, logging, health, erreurs	phase3-api-foundation	✓
4	AuthModule Argon2id + JWT + refresh	phase4-auth	✓
5	RBAC + isolation organisation	phase5-authorization	✓
6	AuditModule hash chain SHA-256	phase6-audit	✓

Réalisations clés :

- Stack Docker Compose (Postgres, Redis, MinIO, Mailpit).
- Format réponse API unifié : `{ data } / { error }`.
- Refresh token atomique (`migration atomic_refresh_token_hash`).
- Chaîne d’audit : chaque `AuditEvent` référence `prevHash`.

4.3.2 Phases 7–18 Ressources métier et queue

TABLE 4.7 – Phases 7 à 18 Modules CRUD et orchestration

Ph.	Livrable	Tests	Statut
7	FilesModule S3/MinIO	phase7-files	✓
8	Organization + Users	phase8-organization-users	✓
9–10	Agents + Vendors	phase9, phase10	✓
11–12	Policy Engine + PoliciesModule	phase11, phase12	✓
13	Vault AES-GCM + Credentials	phase13-credentials-vault	✓
14–15	Workflows + WorkflowRuns	phase14, phase15	✓
16–17	ActionAttempts + Approvals	phase16, phase17	✓
18	QueueModule BullMQ	phase18-queue	✓

4.3.3 Phases 19–29 Worker, workflows et finalisation

TABLE 4.8 – Phases 19 à 29 Worker, browser et workflows SaaS

Ph.	Livrable	Tests	Statut
19–20	Worker foundation + browser-runtime	phase19, phase20	✓
21–22	Vendor sandbox + Connector	phase21, phase22	✓
23	InternalWorkerModule	phase23- internal- worker	✓
24–26	3 workflows SaaS + approbation	phase24– 26	✓
27–28	Receipts + Notifications email	phase27, phase28	✓
29	OpenAPI / Swagger	phase29- api-docs	✓

Workflows implémentés :

- `vendor_invoice_download` : login → téléchargement facture PDF → screenshot → receipt.
- `saas_renewal_check` : login → lecture données renouvellement → screenshot → receipt.
- `plan_downgrade_request` : préparation changement plan → pause approbation → reprise → soumission.

4.4 Extraits de code significatifs

4.4.1 Permissions RBAC partagées

```

1 export const Permission = {
2   AgentCreate: 'agent:create',
3   AgentRead: 'agent:read',
4   CredentialGrant: 'credential:grant',
5   WorkflowRun: 'workflow:run',
6   ApprovalApprove: 'approval:approve',
7   AuditRead: 'audit:read',

```

```
8 // ... 28 permissions au total
9 } as const;
10
11 export const ROLE_PERMISSIONS: Record<UserRole, readonly Permission[]> =
12   {
13     [UserRole.Owner]: PERMISSIONS,
14     [UserRole.Approver]: [
15       Permission.ApprovalRead,
16       Permission.ApprovalApprove,
17       Permission.ReceiptRead,
18       // ...
19     ],
20     [UserRole.Developer]: [
21       ...readPermissions,
22       Permission.WorkflowRun,
23       Permission.WorkflowCancel,
24     ],
25   };
26
```

Listing 4.1 – libs/domain Définition des permissions et rôles

4.4.2 Middleware de sécurité frontend

```
1 export function middleware(request: NextRequest) {
2   const nonce = generateNonce();
3   const response = NextResponse.next();
4
5   // Routes dashboard protegee
6   if (request.nextUrl.pathname.startsWith('/app')) {
7     const session = request.cookies.get('aegisweb_session');
8     if (!session) {
9       return NextResponse.redirect(new URL('/login', request.url));
10    }
11  }
12
13  // CSP avec nonce (pas de unsafe-eval)
14  response.headers.set('Content-Security-Policy', buildCsp(nonce));
15  response.headers.set('X-Frame-Options', 'DENY');
16  return response;
17 }
```

Listing 4.2 – apps/web/middleware.ts Protection routes /app et CSP

4.4.3 Enqueue idempotent BullMQ

```
1 async enqueueWorkflowStart(runId: string, payload: StartPayload) {
2   await this.workflowRunsQueue.add(
```

```
3   'start-workflow-run',
4   payload,
5   {
6     jobId: `start-${runId}`,
7     attempts: 3,
8     backoff: { type: 'exponential', delay: 2000 },
9   },
10 );
11 }
```

Listing 4.3 – Queue Jobs idempotents par runId

4.5 Interfaces du dashboard

Le dashboard Next.js (apps/web) suit une direction UX **evidence-first** : chaque écran opérationnel met en avant les preuves (screenshots, timeline, audit drawer, receipts).

4.5.1 Structure des routes

TABLE 4.9 – Routes principales du dashboard (/app/*)

Route	Fonction
/app/home	Tableau de bord opérationnel
/app/agents	Gestion identités agents
/app/vendors	Fournisseurs SaaS
/app/credentials	Coffre-fort (sans secrets en clair)
/app/policies	Politiques de gouvernance
/app/workflows	Définitions de workflows
/app/runs	Exécutions avec polling état
/app/approvals	File d’approbation humaine
/app/receipts	Reçus de confiance
/app/audit	Journal d’audit chaîné

4.5.2 Captures d'interface

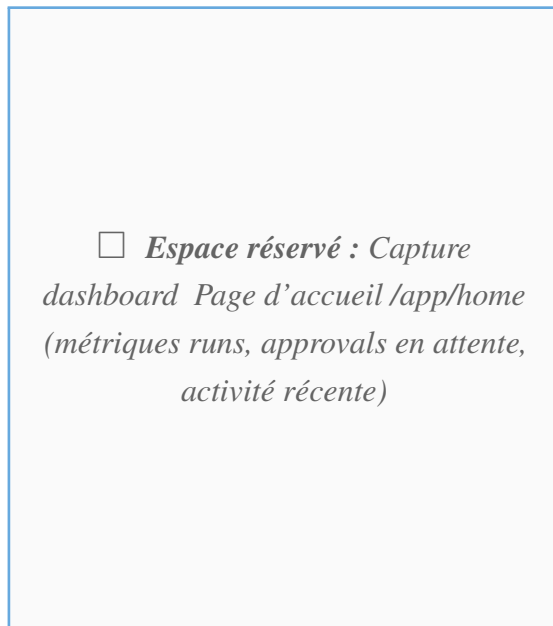


FIGURE 4.1 – Tableau de bord opérationnel

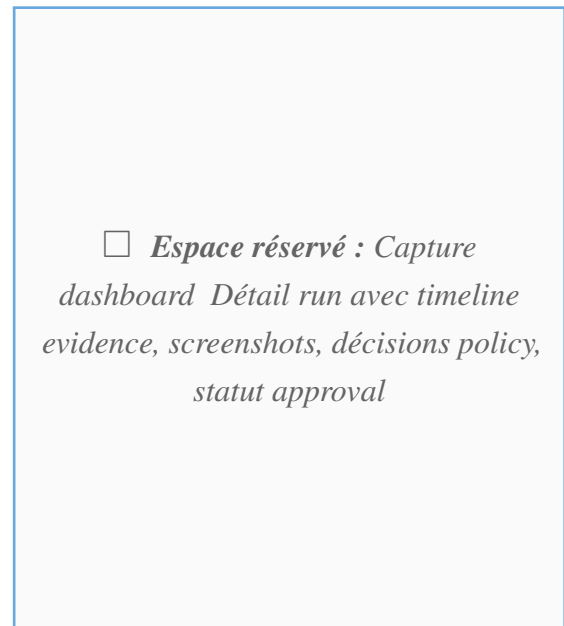


FIGURE 4.2 – Détail d'exécution workflow panneau preuves

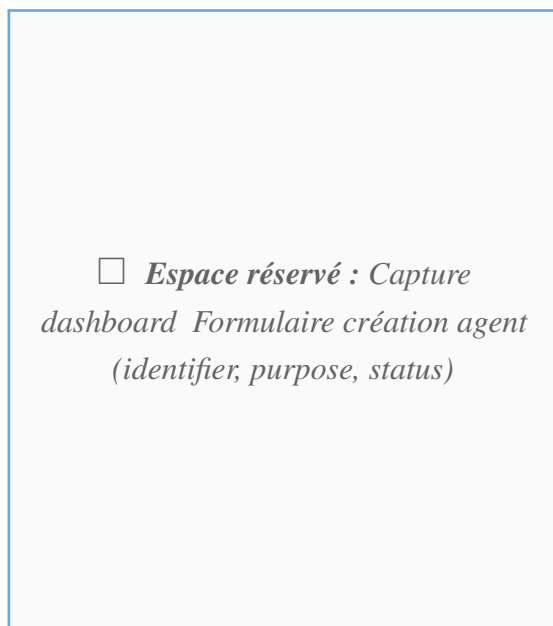


FIGURE 4.3 – Gestion des agents IA

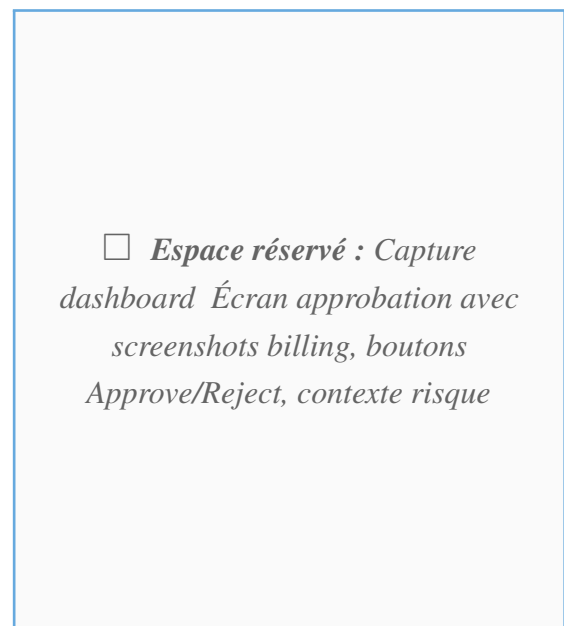


FIGURE 4.4 – Interface d'approbation humaine



FIGURE 4.5 – Logo principal AegisWeb identité visuelle de la plateforme

4.6 Données de démonstration

Après `pnpm db:reset`, l'environnement local contient :

TABLE 4.10 – Données seed Organisation Northstar Labs

Élément	Valeur
Organisation	Northstar Labs (<code>northstarlabs.dev</code>)
Utilisateurs	founder (owner), finance (approver), auditor, dev mot de passe <code>Password123!</code>
Agents	Procurement Bot, Invoice Collector, Risky Admin Bot, Legacy Spend Bot
Vendors	Acme Analytics, Nimbus Docs, Atlas CRM, PayrollPro
Workflows	Acme Invoice Download, Acme Renewal Check, Acme Downgrade Request, Atlas Renewal Check

4.7 Tests et validation

4.7.1 Stratégie de tests

- **33 fichiers Vitest** dans `tests/` (phases 0–29 + acceptance + sécurité + dashboard client).
- Environnement `node` pour backend/worker ; `jsdom` pour composants web.
- Pool forks single-fork pour tests partageant services locaux.
- Scripts complémentaires : `pnpm smoke`, `pnpm e2e:happy`, `pnpm qa:click-path`, `pnpm prod:check`.

4.7.2 Résultats d'acceptation MVP

TABLE 4.11 – Résultats de la suite d'acceptation MVP backend

Métrique	Valeur documentée
Fichiers de tests	31 (+ security + web client)
Tests passés	195
Fichier acceptance	mvp-backend-acceptance.spec.ts
Régression sécurité	security-coverage.spec.ts

4.7.3 Tests fonctionnels représentatifs

TABLE 4.12 – Tests fonctionnels backend représentatifs

ID	Scénario	Résultat attendu	Statut
T01	Login owner Northstar Labs	JWT + cookie refresh	✓
T02	Isolation cross-org	HTTP 404/403	✓
T03	Création agent + grant credential	Grant actif en BDD	✓
T04	Policy deny action interdite	ActionAttempt denied	✓
T05	Workflow invoice download E2E	Run completed + receipt	✓
T06	Downgrade + approbation	Pause → resume → receipt	✓
T07	Audit hash chain integrity	prevHash cohérent	✓
T08	Vault decrypt scopé run	Secret déchiffré worker only	✓
T09	Browser allowlist violation	Action bloquée	✓
T10	Refresh token révoqué	HTTP 401	✓

4.7.4 Score audit sécurité

TABLE 4.13 – Scores d’audit sécurité (docs/SECURITY_AUDIT.md)

Domaine	Score /5
Credential vault	4,0
Auth / sessions	3,8
Autorisation / tenant	4,0
Worker / internal API	3,8
Browser runtime	3,8
Evidence / files	3,3
Frontend	3,8
Pilot readiness global	3,8

4.7.5 Métriques de performance

TABLE 4.14 – Métriques mesurées en environnement de développement local

Métrique	Description	Valeur
Readiness API	GET /health/ready	Postgres + Redis OK
Latence CRUD	GET agents/vendors	< 100 ms typique
Workflow E2E	Downgrade avec approbation	30–90 s (Playwright)
Build Next.js	pnpm build:web	≈ 2–4 min
Suite Vitest complète	pnpm test	≈ 3–8 min

Conclusion du chapitre : Ce chapitre a présenté la réalisation complète d’AegisWeb sur 30 phases incrémentales, couvrant l’API NestJS, le worker Playwright, les bibliothèques de sécurité et le dashboard Next.js. L’intégration PostgreSQL, Redis, BullMQ, MinIO et Prisma a produit un MVP local fonctionnel, validé par 195 tests Vitest et un audit sécurité documenté (3,8/5 pilot readiness).

CONCLUSION GÉNÉRALE

Ce projet de fin d'études nous a offert l'opportunité de concevoir et de développer, de bout en bout, une plateforme de gouvernance pour agents IA agissant sur le web un domaine émergent à l'intersection de la cybersécurité, de l'automatisation et de l'intelligence artificielle.

Bilan du projet

Au terme de ce travail, nous avons conçu et développé **AegisWeb**, une plateforme de confiance pour agents IA web répondant au principe *ñ Permission before autonomy* ž. L'ensemble des objectifs initiaux a été atteint :

- Un **monorepo TypeScript** structuré (4 applications, 7 bibliothèques partagées) avec pnpm workspaces.
- Une **API NestJS** modulaire (20+ modules) avec auth Argon2id, RBAC à 5 rôles et isolation multi-tenant.
- Un **modèle de données Prisma** en 17 tables avec migrations versionnées et seed de démonstration riche.
- Un **coffre-fort AES-256-GCM** et un **moteur de politiques pur** évaluant risques et approbations.
- Un **worker BullMQ + Playwright** exécutant trois workflows SaaS procurement avec preuves et receipts.
- Un **dashboard Next.js 16** evidence-first pour la gestion opérationnelle et l'approbation humaine.
- Une **infrastructure Docker Compose** reproductible et une suite de **195 tests Vitest** validant le MVP.

Sur le plan personnel, ce projet nous a permis de consolider nos compétences en architecture logicielle distribuée, en sécurité applicative (vault, audit chaîné, CSP), en développement fullstack TypeScript (NestJS + Next.js) et en automatisation navigateur contrôlée. Il nous a également sensibilisés aux enjeux de gouvernance des agents IA dans un contexte entreprise.

Difficultés rencontrées

Ce projet n'a pas été exempt de difficultés :

- L'**orchestration worker/API** (tokens HMAC, déchiffrement scopé, reprise après approbation) a exigé une conception rigoureuse des files BullMQ idempotentes.

-
- Le **runtime Playwright contrôlé** (allowlist, blocage réseau privé, masquage screenshots) a nécessité des garde-fous progressifs et des tests dédiés.
 - La **cohérence frontend/backend** (polling runs, refresh token, middleware session) a demandé une couche données React Query structurée.
 - Le **modèle de sécurité global** (redaction secrets, hash audit, isolation org) a requis des tests de régression continus.

Perspectives et améliorations futures

Plusieurs axes d'évolution sont envisagés :

- **Passage production** : checklist pilote (`SECURITY_AUDIT.md`), Postgres/Redis managés, S3 privé, SMTP réel, déploiement Render documenté.
- **BFF HttpOnly** : migration du stockage access token vers cookies HttpOnly pour réduire la surface XSS.
- **mTLS worker** : authentification mutuelle entre worker et API interne en production.
- **Connecteurs vendors réels** : extension au-delà du sandbox vers des portails SaaS réels avec step-up auth.
- **Observabilité** : métriques Prometheus, tracing distribué des runs workflow.
- **Multi-région** : réplication audit et stockage preuves pour conformité réglementaire.

BIBLIOGRAPHIE ET NETOGRAPHIE

Bibliographie

- [1] FOWLER, M., *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2002.
- [2] NEWMAN, S., *Building Microservices*, 2^e édition, O'Reilly Media, 2021.
- [3] MARTIN, R. C., *Clean Architecture : A Craftsman's Guide to Software Structure and Design*, Prentice Hall, 2017.
- [4] RICHARDSON, L. et RUBY, S., *RESTful Web Services*, O'Reilly Media, 2007.
- [5] GAMMA, E. et al., *Design Patterns : Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.
- [6] ANDERSON, R. J., *Security Engineering*, 3^e édition, Wiley, 2020.
- [7] RUBIN, K. S., *Essential Scrum : A Practical Guide to the Most Popular Agile Process*, Addison-Wesley, 2012.

Netographie

- [8] Documentation NestJS, <https://docs.nestjs.com/>, [Consulté le : 15/05/2026].
- [9] Documentation Next.js, <https://nextjs.org/docs>, [Consulté le : 15/05/2026].
- [10] Documentation Prisma, <https://www.prisma.io/docs>, [Consulté le : 10/05/2026].
- [11] Documentation BullMQ, <https://docs.bullmq.io/>, [Consulté le : 12/05/2026].
- [12] Documentation Playwright, <https://playwright.dev/docs/intro>, [Consulté le : 18/05/2026].
- [13] Documentation PostgreSQL 17, <https://www.postgresql.org/docs/17/>, [Consulté le : 08/05/2026].
- [14] OWASP Top Ten 2021, <https://owasp.org/www-project-top-ten/>, [Consulté le : 20/05/2026].
- [15] OWASP ASVS, <https://owasp.org/www-project-application-security-verification/>, [Consulté le : 20/05/2026].
- [16] Documentation TanStack Query, <https://tanstack.com/query/latest>, [Consulté le : 14/05/2026].
- [17] Documentation Docker Compose, <https://docs.docker.com/compose/>, [Consulté le : 05/05/2026].

- [18] Documentation Redis, <https://redis.io/docs/>, [Consulté le : 05/05/2026].
- [19] RFC 9106 Argon2 Memory-Hard Function, <https://www.rfc-editor.org/rfc/rfc9106>, [Consulté le : 10/05/2026].
- [20] NIST SP 800-38D GCM Mode, <https://csrc.nist.gov/publications/detail/sp/800-38d/final>, [Consulté le : 10/05/2026].
- [21] Documentation Vitest, <https://vitest.dev/>, [Consulté le : 22/05/2026].
- [22] Dépôt AegisWeb README et documentation technique, <https://github.com/>, [Consulté le : 22/06/2026].

ANNEXE A

STRUCTURE DU PROJET ARBORESCENCE DES FICHIERS

Listing A.1 – Arborescence du dépôt AegisWeb

```
AegisWeb/
|-- apps/
|   |-- api/                                # API REST NestJS (control plane)
|   |   |-- src/
|   |   |   |-- main.ts
|   |   |   |-- app.module.ts
|   |   |   |-- auth/
|   |   |   |-- authorization/
|   |   |   |-- agents/
|   |   |   |-- vendors/
|   |   |   |-- policies/
|   |   |   |-- credentials/
|   |   |   |-- workflows/
|   |   |   |-- workflow-runs/
|   |   |   |-- approvals/
|   |   |   |-- receipts/
|   |   |   |-- audit/
|   |   |   |-- files/
|   |   |   |-- queue/
|   |   |-- internal-worker/
|   |-- worker/                            # Worker BullMQ + Playwright
|   |-- vendor-sandbox/                    # Portail SaaS simulé
|   |-- web/                               # Dashboard Next.js 16
|   |   |-- app/                           # App Router (/ , /login, /app/*)
|   |   |-- components/                    # UI, product, evidence, brand
|   |   |-- lib/                           # auth, api, data-layer
|   |   |-- middleware.ts
|-- libs/
|   |-- domain/                            # Enums, permissions, types
|   |-- database/                           # Prisma client + seed
|   |-- vault/                              # AES-256-GCM
|   |-- policy-engine/                      # Moteur politiques pur
|   |-- browser-runtime/                   # Playwright contrôlé
|   |-- audit/
|   |-- testing/
|-- prisma/
|   |-- schema.prisma
|   |-- migrations/
|-- infra/
|   |-- docker-compose.yml
|-- tests/                                # 33 specs Vitest (phase0--29)
|-- scripts/                              # demo-local, e2e-happy-path
|-- docs/                                 # Plans phases, audits, runbooks
```

ANNEXE B

RÉFÉRENCE DES ENDPOINTS API

TABLE B.1 – Référence des endpoints API AegisWeb

Méthode	Endpoint	Description	Auth
<i>Authentification (/auth)</i>			
POST	/auth/register	Inscription organisation	Non
POST	/auth/login	Connexion (JWT + refresh cookie)	Non
POST	/auth/refresh	Rafraîchir access token	Cookie
POST	/auth/logout	Révoquer refresh token	JWT
GET	/auth/me	Profil utilisateur courant	JWT
<i>Santé (/health)</i>			
GET	/health	Health check basique	Non
GET	/health/ready	Readiness (Postgres, Redis)	Non
<i>Organisation (/organization)</i>			
GET	/organization	Détails organisation	JWT
PATCH	/organization	Mise à jour organisation	JWT
<i>Utilisateurs (/users)</i>			
GET	/users	Liste membres	JWT
POST	/users/invite	Inviter membre	JWT
PATCH	/users/:id/role	Modifier rôle	JWT
<i>Agents (/agents)</i>			
GET	/agents	Liste agents	JWT

Méthode	Endpoint	Description	Auth
POST	/agents	Créer agent	JWT
GET	/agents/:id	Détail agent	JWT
PATCH	/agents/:id	Modifier agent	JWT
<i>Vendors (/vendors)</i>			
GET/POST	/vendors	CRUD fournisseurs SaaS	JWT
GET/PATCH/DELETE	/vendors/:id	Opérations vendor	JWT
<i>Politiques (/policies)</i>			
GET/POST	/policies	CRUD politiques	JWT
POST	/policies/:id/evaluate	Évaluation politique	JWT
<i>Credentials (/credentials)</i>			
GET/POST	/credentials	CRUD credentials chefs	JWT
POST	/credentials/:id/grants	Accorder à un agent	JWT
<i>Workflows (/workflows)</i>			
GET/POST	/workflows	CRUD définitions workflow	JWT
GET/PATCH	/workflows/:id	Détail / mise à jour	JWT
<i>Exécutions (/workflow-runs)</i>			
GET/POST	/workflow-runs	Liste / lancer run	JWT
GET	/workflow-runs/:id	Détail run + état	JWT
POST	/workflow-runs/:id/cancel	Annuler run	JWT
GET	/workflow-runs/:runId/activities	Tentatives	JWT
<i>Approbations (/approvals)</i>			
GET	/approvals	File d'approbation	JWT
GET	/approvals/:id	Détail approbation	JWT
POST	/approvals/:id/approve	Approuver	JWT
POST	/approvals/:id/reject	Rejeter	JWT

Méthode	Endpoint	Description	Auth
<i>Receipts, Audit, Files</i>			
GET	/receipts	Liste reçus de confiance	JWT
GET	/audit-events	Journal d'audit	JWT
GET	/files/:id	Métadonnées fichier	JWT
<i>Endpoints internes worker (/internal)</i>			
POST	/internal/workers/runs/:id	Événement audit	Worker
POST	/internal/workers/runs/:id	Upload screenshots	Worker
POST	/internal/workers/runs/:id	Fin run/réussie	Worker
POST	/internal/workers/runs/:id	Échec run	Worker
POST	/internal/workers/runs/:id	Demande approbation	Worker
POST	/internal/vault/credentials/:id	Déchiffrement scrypt - for	Worker

ANNEXE C

VARIABLES D'ENVIRONNEMENT

TABLE C.1 – Variables d'environnement principales (`.env.example`)

Variable	Description
DATABASE_URL	Connexion PostgreSQL (agentpass)
REDIS_URL	Connexion Redis pour BullMQ
JWT_ACCESS_SECRET	Secret signature JWT access
JWT_REFRESH_SECRET	Secret refresh tokens
VAULT_MASTER_KEY	Clé maître AES-256-GCM (44 chars base64)
S3_ENDPOINT	/ Stockage MinIO/S3
S3_BUCKET	
API_PORT	/ API (défaut 3001)
API_BASE_URL	
NEXT_PUBLIC_API_URL	URL API pour le frontend
WORKER_INTERNAL_TOKEN	Token worker (dev)
VENDOR_SANDBOX_URL	URL sandbox (défaut 4202)
MAIL_HOST / MAIL_PORT	SMTP Mailpit (1025)
ENABLE_OPENAPI	Activer Swagger (/docs)
NEXT_PUBLIC_ENABLE_DEM	Mode démo dashboard
API_ALLOWED_ORIGINS	CORS origins autorisées

ANNEXE D

COMMANDES DE DÉVELOPPEMENT ET DÉPLOIEMENT

TABLE D.1 – Commandes pnpm principales

Commande	Description
<code>pnpm install</code>	Installation dépendances monorepo
<code>pnpm infra:up</code>	Docker Postgres, Redis, MinIO, Mailpit
<code>pnpm db:reset</code>	Reset BDD + seed démo
<code>pnpm dev</code>	API + worker + web en parallèle
<code>pnpm test</code>	Suite Vitest complète
<code>pnpm smoke</code>	Vérification services locaux
<code>pnpm e2e:happy</code>	Parcours E2E heureux
<code>pnpm demo:local</code>	Démo locale one-command
<code>pnpm prod:check</code>	typecheck + lint + build
<code>pnpm</code> <code>stack:hybrid:up</code>	Backend Docker + frontend local

TABLE D.2 – URLs de services en développement local

Service	URL
Dashboard	http://localhost:3000
API	http://localhost:3001
Readiness	http://localhost:3001/health/ready
Swagger	http://localhost:3001/docs
Vendor sandbox	http://localhost:4202
Mailpit	http://localhost:8025
MinIO console	http://localhost:9001